

Clear and Easy Demonstration of Database Normalization

1NF, 2NF, 3NF, BCNF, 4NF and 5NF with Step-by-Step Examples

Instructor: Md Mynoddin
Assistant Professor (CSE), RMSTU

CSE 2201: Database Management Systems

Learning Outcomes

By the end of this lecture, students should be able to:

- explain why normalization is needed in relational database design;
- identify functional dependencies, partial dependencies, and transitive dependencies;
- convert an unnormalized table into 1NF, 2NF, 3NF and BCNF;
- recognize multivalued dependencies and apply 4NF;
- understand the idea of join dependency and 5NF;
- decide the suitable normal form for practical database projects.

The Big Idea of Normalization

Simple definition

Normalization is the process of organizing data into well-structured tables so that the database avoids unnecessary repetition, update problems, and inconsistent data.

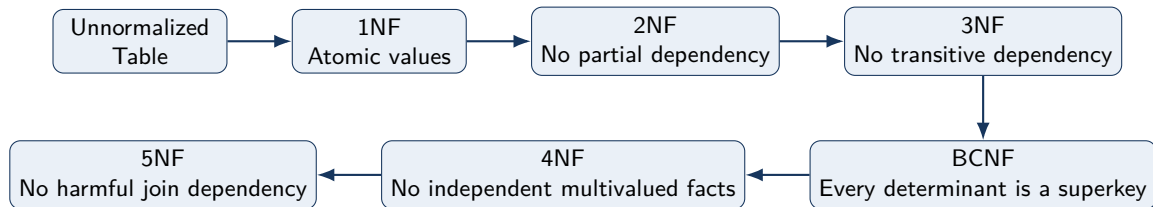
Without normalization

- repeated values
- mixed topics in one table
- update anomaly
- insertion anomaly
- deletion anomaly

With normalization

- one table stores one main fact type
- less redundancy
- clear primary and foreign keys
- better data consistency
- easier maintenance

Normalization Roadmap



Practical observation

Most undergraduate database projects should aim for **3NF** or **BCNF**. 4NF and 5NF are less common, but important for understanding advanced design problems.

Before Normalization: Keys and Dependencies

Key terms

- **Relation:** table
- **Tuple:** row
- **Attribute:** column
- **Candidate key:** minimal attribute set that uniquely identifies a row
- **Primary key:** chosen candidate key

Example: $StudentID \rightarrow StudentName, DeptID$

Functional dependency

Notation

$$X \rightarrow Y$$

It means: if two rows have the same value of X , they must have the same value of Y .

Example of Functional Dependency

StudentID	StudentName	DeptID	DeptName
2201	Nowshin Tasnim	CSE	Computer Science
2202	Avijit Dutta	CSE	Computer Science
2401	Arif Hossain	EEE	Electrical Engineering

Valid dependencies

- $StudentID \rightarrow StudentName$
- $StudentID \rightarrow DeptID$
- $DeptID \rightarrow DeptName$

Important insight

$StudentID \rightarrow DeptName$ is true, but indirectly:

$$StudentID \rightarrow DeptID \rightarrow DeptName$$

This is a **transitive dependency**.

Three Common Anomalies

Anomaly	Meaning	Example
Update anomaly	Same fact must be changed in many rows	Department name is repeated for every student; changing it requires many updates.
Insertion anomaly	Cannot insert one fact without another unrelated fact	Cannot add a new department until at least one student exists.
Deletion anomaly	Deleting one row accidentally removes another important fact	Deleting the last student of a department removes department information.

First Normal Form (1NF)

Rule

A relation is in **First Normal Form** if every cell contains only **one atomic value**; no repeating groups or list-valued attributes are allowed.

Ask this question:

Is any cell storing a list, set, or repeated group?

If yes

Separate the repeating values into rows or a new table.

Not in 1NF: Repeating Values

StudentID	StudentName	Phones	Courses
2201	Nowshin	01711, 01822	CSE1101, CSE2201
2202	Avijit	01933	CSE1101, CSE1201

Problems

- The attribute Phones contains multiple values in one cell.
- The attribute Courses contains multiple values in one cell.
- Searching and joining become difficult.

1NF Solution: Make Values Atomic

StudentPhone

StudentID	Phone
2201	01711
2201	01822
2202	01933

Key: {*StudentID*, *Phone*}

StudentCourse

StudentID	CourseID
2201	CSE1101
2201	CSE2201
2202	CSE1101
2202	CSE1201

Key: {*StudentID*, *CourseID*}

Result

Every cell now contains a single value. The tables are in 1NF.

Second Normal Form (2NF)

Rule

A relation is in **2NF** if it is in 1NF and every non-key attribute depends on the **whole candidate key**, not only part of a composite key.

2NF mainly matters when the primary key is composite.

Composite key? Check for partial dependency.

Not in 2NF: Partial Dependency

Relation: Enrollment(StudentID, CourseID, StudentName, CourseTitle, Grade)

Primary key: {StudentID, CourseID}

StudentID	CourseID	StudentName	CourseTitle	Grade
2201	CSE2201	Nowshin	Database Systems	A
2201	CSE2203	Nowshin	Data Structures	A-
2202	CSE2201	Avijit	Database Systems	B+

Partial dependencies

- *StudentID* → *StudentName* depends only on part of the key.
- *CourseID* → *CourseTitle* depends only on part of the key.
- *StudentID, CourseID* → *Grade* depends on the full key.

2NF Solution: Decompose Partial Dependencies

<u>Student</u>		<u>Course</u>		<u>Enrollment</u>		
StudentID	StudentName	CourseID	CourseTitle	StudentID	CourseID	Grade
2201	Nowshin	CSE2201	Database Systems	2201	CSE2201	A
2202	Avijit			2201	CSE2203	A-
		CSE2203	Data Structures	2202	CSE2201	B+

Key lesson

Attributes about students go to Student. Attributes about courses go to Course. The many-to-many relationship data remains in Enrollment.

Third Normal Form (3NF)

Rule

A relation is in **3NF** if it is in 2NF and there is no transitive dependency of a non-key attribute on the key.

$\text{Key} \rightarrow \text{non-key} \rightarrow \text{non-key}$ is not allowed.

In simple words: A table should not store facts about another non-key attribute.

Not in 3NF: Transitive Dependency

Relation: Student(StudentID, StudentName, DeptID, DeptName, Building)

StudentID	StudentName	DeptID	DeptName	Building
2201	Nowshin	CSE	Computer Science	Academic A
2202	Avijit	CSE	Computer Science	Academic A
2401	Arif	EEE	Electrical Engineering	Academic B

Functional dependencies

- $StudentID \rightarrow StudentName, DeptID$
- $DeptID \rightarrow DeptName, Building$

Therefore:

$$StudentID \rightarrow DeptID \rightarrow DeptName, Building$$

This violates 3NF.

3NF Solution: Separate Department Facts

Student

StudentID	StudentName	DeptID
2201	Nowshin	CSE
2202	Avijit	CSE
2401	Arif	EEE

Foreign key: DeptID

Department

DeptID	DeptName	Building
CSE	Computer Science	Academic A
EEE	Electrical Engineering	Academic B

Primary key: DeptID

Result

Student facts and department facts are now stored separately. This removes transitive dependency and update anomaly.

1NF, 2NF and 3NF: Quick Difference

Normal Form	Problem removed	Question to ask
1NF	Repeating groups and non-atomic values	Does every cell contain one value only?
2NF	Partial dependency on part of composite key	Does every non-key attribute depend on the whole key?
3NF	Transitive dependency	Does a non-key attribute depend on another non-key attribute?

Simple memory aid

1NF: one value per cell. **2NF:** whole key. **3NF:** nothing but the key.

Boyce-Codd Normal Form (BCNF)

Rule

A relation is in **BCNF** if for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey.

Every determinant must be a candidate key or superkey.

Why BCNF after 3NF?

Some relations may satisfy 3NF but still have redundancy when a non-key determinant determines a key attribute.

BCNF Example: 3NF but not BCNF

Relation: StudentInstructor(StudentID, InstructorID, CourseID)

Business rules:

- A student takes a course from one instructor.
- Each instructor teaches exactly one course.

Functional dependencies:

- $StudentID, CourseID \rightarrow InstructorID$
- $InstructorID \rightarrow CourseID$

StudentID	InstructorID	CourseID
2201	I10	DBMS
2202	I10	DBMS
2203	I20	OS

BCNF violation

$InstructorID \rightarrow CourseID$, but InstructorID is not a superkey of this table.

BCNF Solution

InstructorCourse

InstructorID	CourseID
I10	DBMS
I20	OS

Key: InstructorID

StudentInstructor

StudentID	InstructorID
2201	I10
2202	I10
2203	I20

Key: {StudentID, InstructorID}

Result

The dependency $InstructorID \rightarrow CourseID$ is now stored in a table where InstructorID is the key. Therefore, the design satisfies BCNF.

3NF vs BCNF

Issue	3NF	BCNF
Main rule	Allows some dependencies where the right side is a prime attribute	Requires every determinant to be a superkey
Strictness	Less strict	More strict
Dependency preservation	Usually easier to preserve	Sometimes decomposition may lose dependency preservation
Practical use	Common target for design	Stronger target when possible

Teaching summary

3NF is often enough for practical designs. BCNF is preferred when the decomposition is lossless and dependency issues are manageable.

Fourth Normal Form (4NF)

Rule

A relation is in **4NF** if it is in BCNF and has no non-trivial multivalued dependency except those determined by a superkey.

Multivalued dependency notation:

$$X \twoheadrightarrow Y$$

It means: for each value of X , there is an independent set of values of Y .

Common symptom

One table combines two independent many-valued facts about the same entity.

Not in 4NF: Independent Multivalued Facts

Relation: StudentSkillLanguage(StudentID, Skill, Language)

Meaning: A student may have many skills and may know many languages. Skills and languages are independent facts.

StudentID	Skill	Language
2201	Java	Bangla
2201	Java	English
2201	Python	Bangla
2201	Python	English

Problem

All combinations of skills and languages must be stored. This creates unnecessary rows.

4NF Solution: Separate Independent Multivalued Facts

StudentSkill

StudentID	Skill
2201	Java
2201	Python

MVD represented: $StudentID \twoheadrightarrow Skill$

StudentLanguage

StudentID	Language
2201	Bangla
2201	English

MVD represented: $StudentID \twoheadrightarrow Language$

Result

No artificial combinations are stored. Each table stores one independent multivalued fact.

Fifth Normal Form (5NF)

Rule

A relation is in **5NF** if every non-trivial join dependency is implied by candidate keys.

In simple words: A table should not store a complex many-way relationship if it can be reconstructed without loss from smaller, meaningful relationships.

Practical note

5NF is rare in small applications. It becomes relevant when a relationship among three or more entities is actually determined by pairwise relationships.

5NF Example: Supplier-Part-Project

Relation: Supply(Supplier, Part, Project)

Assume a supplier can supply a part to a project if these three pairwise facts are all true:

- Supplier supplies the part.
- Supplier works with the project.
- Part is used in the project.

Original table

S	P	J
S1	P1	J1
S1	P2	J1
S2	P1	J1

Possible 5NF decomposition

- SupplierPart(S, P)
- SupplierProject(S, J)
- PartProject(P, J)

Important

Decompose to 5NF only if the original relation can be exactly reconstructed by joining the smaller tables.

All Normal Forms in One Table

NF	Removes	Simple test
1NF	Repeating groups, lists, non-atomic values	Are all cells atomic?
2NF	Partial dependency	Does every non-key attribute depend on the whole key?
3NF	Transitive dependency	Does any non-key attribute determine another non-key attribute?
BCNF	Non-key determinant	Is every determinant a superkey?
4NF	Independent multivalued dependencies	Are two independent multi-valued facts mixed in one table?
5NF	Join dependency	Can a complex table be reconstructed from smaller meaningful joins?

Complete Example: Start with One Bad Table

Bad table: StudentCourseInfo

SID	SName	Phones	CID	CTitle	Instructor	Grade	DeptName
2201	Nowshin	017,018	CSE2201	DBMS	Dr. Rahman	A	CSE
2201	Nowshin	017,018	CSE2203	DSA	Mr. Hasan	A-	CSE
2202	Avijit	019	CSE2201	DBMS	Dr. Rahman	B+	CSE

Problems

- Phones contains repeating values: violates 1NF.
- Student name depends only on SID: partial dependency.
- Course title depends only on CID: partial dependency.
- Department name depends on student's department, not on enrollment.

Step 1: Convert to 1NF

Move multi-valued phone numbers to a separate table.

StudentPhone

SID	Phone
2201	017
2201	018
2202	019

EnrollmentInfo in 1NF

SID	SName	CID	CTitle	Grade
2201	Nowshin	CSE2201	DBMS	A
2201	Nowshin	CSE2203	DSA	A-
2202	Avijit	CSE2201	DBMS	B+

Now 1NF holds

Every cell contains one value. But redundancy still remains.

Step 2: Convert to 2NF

Original key in enrollment data: $\{SID, CID\}$

Partial dependencies:

- $SID \rightarrow SName$
- $CID \rightarrow CTitle$
- $SID, CID \rightarrow Grade$

Student

Student(SID, SName)

Course

Course(CID, CTitle)

Enrollment

Enrollment(SID, CID, Grade)

Now 2NF holds

Every non-key attribute depends on the whole key of its table.

Step 3: Convert to 3NF

Now suppose Student still has department data:

Student(SID, SName, DeptID, DeptName)

Transitive dependency

$$SID \rightarrow DeptID \rightarrow DeptName$$

Student

Student(SID, SName, DeptID)

DeptID becomes a foreign key.

Department

Department(DeptID, DeptName)

DeptID is the primary key.

Now 3NF holds

Department information is no longer repeated in every student row.

Final Normalized Design for the Example

Table	Purpose
Student(SID, SName, DeptID)	Stores student facts only.
StudentPhone(SID, Phone)	Stores multiple phone numbers separately.
Department(DeptID, DeptName)	Stores department facts only.
Course(CID, CTitle)	Stores course facts only.
Enrollment(SID, CID, Grade)	Stores the relationship between student and course.
InstructorCourse(InstructorID, CID)	Stores which instructor teaches which course, if needed.

Final idea

Each table now has one main responsibility. This is the practical heart of normalization.

SQL Implementation: 3NF Tables

```
CREATE TABLE department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(80) NOT NULL UNIQUE  
);  
  
CREATE TABLE student (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(80) NOT NULL,  
    dept_id INT NOT NULL,  
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)  
);  
  
CREATE TABLE course (  
    course_id VARCHAR(10) PRIMARY KEY,  
    course_title VARCHAR(120) NOT NULL  
);  
  
CREATE TABLE enrollment (  
    student_id INT NOT NULL,  
    course_id VARCHAR(10) NOT NULL,  
    grade VARCHAR(2),  
    PRIMARY KEY (student_id, course_id),  
    FOREIGN KEY (student_id) REFERENCES student(student_id),  
    FOREIGN KEY (course_id) REFERENCES course(course_id)  
);
```

SQL Query After Normalization

Even though data is separated, we can reconstruct meaningful information using joins.

```
SELECT
    s.student_id,
    s.student_name,
    d.dept_name,
    c.course_title,
    e.grade
FROM enrollment e
JOIN student s
    ON s.student_id = e.student_id
JOIN department d
    ON d.dept_id = s.dept_id
JOIN course c
    ON c.course_id = e.course_id
ORDER BY s.student_id, c.course_id;
```

Key point

Normalization decomposes tables for correctness. JOIN reconstructs reports for users.

Class Practice 1: Identify the Normal Form Problem

Relation: BookLoan(MemberID, MemberName, BookID, BookTitle, AuthorName, IssueDate)

Primary key: {*MemberID, BookID, IssueDate*}

Dependencies:

- *MemberID* → *MemberName*
- *BookID* → *BookTitle, AuthorName*
- *MemberID, BookID, IssueDate* → *ReturnDate* if return date exists

Question

Which normal form is violated and how should we decompose the relation?

Solution to Practice 1

Violated normal form: 2NF, because non-key attributes depend on part of the composite key.

Decomposition:

- Member(MemberID, MemberName)
- Book(BookID, BookTitle, AuthorName)
- Loan(MemberID, BookID, IssueDate, ReturnDate)

Why this works

Member information is stored once. Book information is stored once. Loan records store only borrowing facts.

Class Practice 2: 3NF Problem

Relation: Employee(EmpID, EmpName, DeptID, DeptName, DeptLocation)

Dependencies:

- $EmpID \rightarrow EmpName, DeptID$
- $DeptID \rightarrow DeptName, DeptLocation$

Question

Identify the transitive dependency and decompose the relation into 3NF.

Solution to Practice 2

Transitive dependency:

$EmpID \rightarrow DeptID \rightarrow DeptName, DeptLocation$

3NF decomposition:

- Employee(EmpID, EmpName, DeptID)
- Department(DeptID, DeptName, DeptLocation)

Benefit

If department location changes, update only one row in Department, not every employee row.

Decision Tree for Students

1. Does any cell contain a list or repeating group?
Yes $\Rightarrow$ fix 1NF.
2. Is there a composite key?
Yes $\Rightarrow$ check partial dependencies for 2NF.
3. Does a non-key attribute determine another non-key attribute?
Yes $\Rightarrow$ fix 3NF.
4. Does any determinant fail to be a superkey?
Yes $\Rightarrow$ check BCNF.
5. Are two independent many-valued facts stored together?
Yes $\Rightarrow$ fix 4NF.
6. Is a complex relationship exactly reconstructable from smaller joins?
Yes $\Rightarrow$ check 5NF.

Common Student Mistakes

Mistake	Correction
Thinking normalization means splitting every column into a new table.	Split only when dependencies or repeating values justify it.
Ignoring the primary key before checking 2NF.	Always identify candidate keys first.
Confusing 2NF and 3NF.	2NF is about partial dependency; 3NF is about transitive dependency.
Forgetting that normalized tables need joins.	Joins are normal and expected after decomposition.
Removing too much and losing information.	Decomposition must be lossless.

Final Summary

NF	One-sentence memory
1NF	Store one value in each cell.
2NF	Every non-key attribute depends on the whole key.
3NF	Non-key attributes should not depend on other non-key attributes.
BCNF	Every determinant must be a superkey.
4NF	Do not mix independent multi-valued facts.
5NF	Do not store a many-way relationship if it is fully reconstructable from smaller joins.

Practical target

For most CSE 2201 project work: design up to 3NF or BCNF, and understand 4NF/5NF conceptually.

Homework / Lab Task

Choose one table from your SQL project assignment and write:

1. the original relation schema;
2. the primary key and candidate keys;
3. all meaningful functional dependencies;
4. whether the table is in 1NF, 2NF and 3NF;
5. the decomposed tables if normalization is needed;
6. SQL CREATE TABLE statements for the normalized design.

Viva preparation

You should be able to explain why each table was created and which dependency was removed.